

1 MARCO TEÓRICO

1.1 Ingeniería y Desarrollo Dirigidos por Modelos

La Ingeniería Dirigida por Modelos (Model Driven Engineering, MDE) es un enfoque de desarrollo de software en el cual los modelos constituyen artefactos principales del proceso. A diferencia del modelado utilizado únicamente como documentación, en MDE los modelos contienen información estructurada que puede ser procesada mediante mecanismos automáticos de transformación [1].

El Desarrollo Dirigido por Modelos (Model Driven Development, MDD) representa la aplicación práctica de estos principios. En este enfoque, el sistema se describe mediante modelos formales que posteriormente se transforman en código fuente u otros artefactos de implementación. Su finalidad es aumentar el nivel de abstracción, reducir tareas repetitivas y mantener la trazabilidad entre el diseño y el software generado [2].

Aunque MDD facilita la automatización del desarrollo, todavía presenta desafíos relacionados con la interoperabilidad de herramientas, la evolución de los modelos, la mantenibilidad y su adopción en proyectos reales [3].

1.2 Arquitectura Dirigida por Modelos

La Arquitectura Dirigida por Modelos (Model Driven Architecture, MDA) es una propuesta del Object Management Group que organiza el desarrollo mediante diferentes niveles de abstracción [4].

Los principales niveles son:

- **Modelo Independiente de Computación (CIM):** representa los procesos del negocio y las necesidades de los usuarios sin considerar aspectos tecnológicos.
- **Modelo Independiente de Plataforma (PIM):** describe la estructura y el comportamiento del sistema sin depender de una plataforma específica.
- **Modelo Específico de Plataforma (PSM):** incorpora las características tecnológicas necesarias para implementar el sistema en una plataforma determinada.

En el proyecto MundiPets, el diagrama de clases UML representa la estructura del subsistema y se utiliza como modelo de origen para generar código Java mediante Java Designer.

1.3 UML, perfiles y estereotipos

El Lenguaje Unificado de Modelado (Unified Modeling Language, UML) es un estándar utilizado para especificar, visualizar y documentar sistemas de software. Sus diagramas permiten representar clases, atributos, operaciones, relaciones y multiplicidades dentro de una solución orientada a objetos [5].

Los perfiles UML permiten adaptar el lenguaje a necesidades tecnológicas específicas mediante estereotipos y valores etiquetados. En MundiPets se empleó el estereotipo:

«JavaClass»

Este estereotipo, proporcionado por Java Designer 5.4.00, permite identificar las clases que deben transformarse en archivos Java. No se utilizaron estereotipos de persistencia, servicios o controladores, porque el generador empleado produce principalmente estructuras de clases Java.

Los modelos y perfiles se sustentan en metamodelos que definen las reglas de construcción de los elementos utilizados. El estándar Meta Object Facility proporciona una infraestructura para definir metamodelos y facilitar la interoperabilidad entre herramientas dirigidas por modelos [6].

1.4 Transformaciones y generación de código

Las transformaciones de modelos permiten convertir una representación del sistema en otra.

Dentro de MDD se distinguen principalmente dos tipos:

- **Modelo a Modelo (M2M):** transforma un modelo de origen en otro modelo con diferente nivel de abstracción. Un ejemplo es la transformación de un PIM hacia un PSM. El estándar QVT proporciona mecanismos para definir este tipo de transformaciones [7].
- **Modelo a Texto (M2T):** transforma elementos de un modelo en archivos textuales, como código fuente. El estándar MOFM2T establece mecanismos para generar texto a partir de modelos basados en MOF [8].

La generación realizada en MundiPets corresponde a una transformación M2T, debido a que Java Designer interpreta las clases, atributos, operaciones, asociaciones y multiplicidades del modelo UML para producir archivos con extensión .java.

La ingeniería directa describe el proceso de generar código desde un modelo. Por otra parte, el roundtrip controlado permite modificar el modelo y regenerar el código para comprobar que el cambio se refleja en el artefacto correspondiente. En este proyecto se añadió el atributo microchip a la clase Mascota y posteriormente se regeneró Mascota.java.

La generación automática disminuye parte de la codificación repetitiva, pero no garantiza por sí sola una aplicación completa. La lógica de negocio, la persistencia, la interfaz gráfica y otros componentes deben ser implementados manualmente. Por ello, el código generado debe complementarse con procesos de validación, compilación y prueba.

2 SELECCIÓN DE LA HERRAMIENTA MDD

2.1 Herramienta seleccionada

Para el desarrollo del proyecto MundiPets se seleccionó Modelio 5.4.01 como entorno de modelado UML y Java Designer 5.4.00 como módulo de generación de código Java.

De acuerdo con la documentación oficial, Modelio es un entorno de modelado orientado al desarrollo de software que proporciona soporte para UML, asistencia durante el modelado, comprobación de consistencia y una arquitectura extensible mediante módulos [9].

Dentro de esta arquitectura, Java Designer permite generar código Java desde un modelo UML, recuperar información desde código existente, generar documentación Javadoc y automatizar determinadas actividades relacionadas con el desarrollo Java [10].

La versión utilizada corresponde a la distribución de código abierto de Modelio, publicada bajo la licencia GNU General Public License versión 3.0 (GPL-3.0) [11]. Por esta razón, su utilización académica no requirió adquirir una licencia comercial, lo cual favoreció la reproducibilidad de la demostración por parte de los integrantes y del docente.

2.2 Características principales

Las principales características de Modelio relacionadas con el desarrollo dirigido por modelos son:

- **Soporte para UML:** permite elaborar diagramas estructurales y de comportamiento utilizando el estándar UML definido por el Object Management Group (OMG), facilitando la representación formal del sistema mediante clases, atributos, operaciones y relaciones [5].
- **Generación automática de código:** mediante módulos especializados permite transformar elementos del modelo UML en estructuras iniciales de código fuente, aplicando transformaciones modelo-a-texto dentro del proceso MDD [8].
- **Uso de estereotipos UML:** permite extender los elementos del modelo mediante información adicional asociada a perfiles UML, facilitando la adaptación del modelo hacia plataformas específicas [5].
- **Ingeniería directa e inversa:** permite establecer mecanismos para generar código desde modelos y recuperar modelos desde código existente, favoreciendo la evolución y mantenimiento del software [2]

- Arquitectura modular: incorpora módulos adicionales que amplían las capacidades de la herramienta, permitiendo incorporar funcionalidades de generación, validación y personalización del proceso de desarrollo.

2.3 Justificación de la herramienta

La selección de Modelio 5.4.01 con Java Designer 5.4.00 se realizó considerando cinco criterios: soporte UML, generación de código Java, validación del modelo, licencia y facilidad de reproducción del proceso.

Modelio permite construir diagramas UML y ejecutar comprobaciones de consistencia sobre los elementos del modelo. Estas funciones fueron necesarias para representar las clases, atributos, operaciones, asociaciones y multiplicidades del subsistema MundiPets antes de la generación del código [9].

Java Designer fue seleccionado porque constituye el módulo específico de Modelio para trabajar con Java. La documentación oficial señala que este módulo soporta generación de código, ingeniería inversa, documentación Javadoc y automatización Java [10]. En la demostración se utilizó su configuración estándar para transformar las clases UML marcadas con el estereotipo «JavaClass» en archivos con extensión .java.

Otro criterio determinante fue el licenciamiento. Modelio y sus extensiones de código abierto se distribuyen bajo la licencia GPL-3.0 [11]. En consecuencia, el equipo pudo instalar, utilizar y documentar la herramienta sin adquirir una licencia comercial, condición apropiada para un proyecto académico y para la reproducción posterior de la demostración.

Frente a las alternativas analizadas, Modelio ofreció una combinación adecuada de modelado UML, auditoría, generación Java y disponibilidad sin costo de licencia. Enterprise Architect y determinadas funciones de Visual Paradigm dependen de ediciones comerciales, mientras que Papyrus con Acceleo permite mayor personalización, pero requiere una configuración más compleja. Por estas razones, Modelio fue la alternativa más adecuada para demostrar el flujo UML → Java Designer → código Java dentro del alcance de MundiPets.

2.4 Comparación con herramientas alternativas

La selección de una herramienta MDD requiere analizar diferentes alternativas considerando aspectos como soporte UML, capacidad de transformación, generación de código, licencia y facilidad de uso. La siguiente comparación permite justificar la elección realizada.

Herramienta	Ventajas	Limitaciones
Modelio 5.4.01	Soporte UML, auditoría del modelo, arquitectura modular, Java Designer para generación de código Java y distribución de código abierto bajo licencia GPL-3.0 [9-11].	El generador estándar produce principalmente estructuras de clases; la lógica de negocio y otras funcionalidades deben completarse manualmente.

Enterprise Architect	Amplio soporte MDA, múltiples lenguajes de programación y generación de código mediante plantillas.	Requiere licencia comercial para varias funcionalidades avanzadas.
Visual Paradigm	Amplias capacidades de modelado empresarial e integración tecnológica, con soporte para diferentes diagramas UML.	Varias funciones relacionadas con generación y transformación dependen de versiones comerciales.
Papyrus + Aceleo	Alta personalización mediante estándares OMG y soporte para transformaciones modelo a texto (M2T).	Mayor complejidad de configuración y aprendizaje inicial.

Después del análisis comparativo, Modelio fue seleccionado debido a que proporciona las funcionalidades necesarias para cumplir los objetivos del proyecto MundiPets, permitiendo desarrollar el flujo completo de Ingeniería Dirigida por Modelos desde la construcción del modelo UML hasta la generación del código fuente.

3 DESCRIPCIÓN DEL SISTEMA MUNDIPETS

El proyecto MundiPets responde a una necesidad frecuente en el desarrollo tradicional de software: la separación entre las etapas de diseño e implementación, que suele producir inconsistencias entre los modelos documentados y el código construido manualmente. Para reducir esta brecha, el proyecto aplica el enfoque Model Driven Development (MDD), utilizando un modelo UML como fuente autoritativa a partir de la cual se genera y regenera el código fuente de forma trazable.

3.1 Descripción general del sistema

MundiPets es una plataforma digital orientada a la gestión responsable de mascotas, cuyo propósito es facilitar la administración de información relacionada con animales de compañía, procesos de adopción, seguimiento sanitario y comunicación entre los diferentes actores involucrados.

El sistema permite centralizar información relevante de las mascotas mediante el registro de datos generales, historial médico, vacunas, documentos veterinarios y antecedentes asociados. De esta manera, proporciona una estructura organizada para la consulta y gestión de información necesaria durante los procesos de cuidado, adopción y seguimiento veterinario.

Desde la perspectiva del Desarrollo Dirigido por Modelos (Model Driven Development, MDD), MundiPets utiliza modelos UML como elementos principales para representar la estructura del sistema antes de la generación del código fuente. El modelo de clases desarrollado permite definir las entidades principales del dominio, sus atributos, operaciones

y relaciones, sirviendo como base para el proceso de transformación automática mediante la herramienta seleccionada.

3.2 Alcance del módulo desarrollado

Para la demostración del proceso MDD se desarrolló el módulo de gestión de información de mascotas y control veterinario, debido a que permite representar adecuadamente los elementos principales del dominio y evidenciar la transformación desde un modelo UML hacia código fuente.

El módulo contempla las siguientes funcionalidades:

- Registro de información básica de mascotas, incluyendo nombre, raza, sexo, fecha de nacimiento y estado de salud.
- Administración del historial médico, permitiendo almacenar información relacionada con enfermedades, tratamientos y antecedentes clínicos.
- Gestión de vacunas mediante el registro de nombre de vacuna, fecha de aplicación y fecha de vencimiento.
- Gestión de documentos veterinarios asociados a la mascota, permitiendo almacenar y validar información médica.
- Asociación entre mascotas y profesionales veterinarios para la validación de información sanitaria

El alcance del desarrollo se enfoca en la construcción del modelo estructural del sistema mediante UML y su posterior transformación hacia código fuente utilizando la herramienta Modelio 5.4 con el módulo Java Designer.

3.3 Actores del subsistema

Los actores directamente relacionados con el subsistema de gestión de información de mascotas y control veterinario son:

- **Propietario:** usuario responsable de registrar, actualizar y consultar la información general y sanitaria de las mascotas asociadas a su perfil.
- **Veterinario:** profesional encargado de consultar información médica, revisar documentos veterinarios y registrar validaciones relacionadas con la salud de las mascotas.

La clase Mascota constituye la entidad principal del dominio, pero no representa un actor externo del sistema.

Estos requisitos funcionales representan la base para la construcción del modelo UML del sistema MundiPets, donde cada elemento del dominio es representado mediante clases, atributos, métodos y relaciones. Posteriormente, el modelo será utilizado como entrada para el proceso de generación automática de código dentro del enfoque MDD.

5 CONFIGURACIÓN Y GENERACIÓN DEL CÓDIGO FUENTE

5.1 Configuración del generador

La transformación del modelo UML hacia código fuente se realizó mediante Java Designer 5.4.00, módulo incorporado en Modelio 5.4.01, compilación 202312071609, para arquitectura Windows x86_64.

Se utilizó el generador estándar proporcionado por Java Designer. Esta decisión no respondió a una omisión del proceso de configuración. En la instalación utilizada, el módulo no expone desde su interfaz archivos de plantilla editables por el usuario. Por tanto, no se efectuó una personalización artificial que no pudiera reproducirse posteriormente.

La configuración se controló mediante los elementos que Java Designer interpreta directamente desde el modelo: estereotipos, tipos de datos, firmas de operaciones, multiplicidades, nombres de rol y visibilidad. Para identificar las clases generables se aplicó el estereotipo:

«JavaClass»

La configuración utilizada se resume en la Tabla

Parámetro	Configuración aplicada
Entorno de modelado	Modelio 5.4.01.
Generador	Java Designer 5.4.00, generador estándar del módulo.
Lenguaje objetivo	Java.
Estereotipo de generación	«JavaClass»
Directorio de salida	C:/Users/USER/modelio/workspace/Ing/src
Convención de nombres	Cada archivo conserva el nombre de su clase UML y utiliza la extensión .java.
Fuente autoritativa	El modelo UML construido en Modelio.
Política de regeneración	Los cambios estructurales se corrigen en el modelo y después se regenera el archivo correspondiente.
Código manual	Se mantiene separado del código generado; únicamente Main.java fue escrito manualmente.

Debido a que no se personalizaron plantillas, se creó el archivo plantillas/configuracion-java-designer.md dentro del repositorio. Este archivo registra la versión del generador, los parámetros efectivos, las convenciones de nombres, las decisiones de mapeo, la política de regeneración y las limitaciones observadas. De esta manera, la configuración puede ser revisada y reproducida sin atribuir al módulo capacidades de edición que no fueron utilizadas.

5.2 Procedimiento de generación

La generación se ejecutó individualmente sobre las clases seleccionadas mediante el siguiente procedimiento:

1. Abrir el proyecto MundiPets en Modelio 5.4.01 y comprobar que Java Designer 5.4.00 estuviera habilitado.
2. Verificar la aplicación del estereotipo «JavaClass» en las clases destinadas a generación.
3. Seleccionar cada clase y ejecutar la opción Java Designer → Generate.
4. Comprobar la creación de los archivos .java en el directorio src y revisarlos mediante Visual Studio Code.

Las capturas de la habilitación del módulo, la aplicación del estereotipo, la ejecución del generador y los archivos obtenidos se presentan en los anexos.

El modelo UML se mantuvo como fuente principal de la estructura del código. Los errores relacionados con atributos, tipos, parámetros, operaciones, multiplicidades o nombres de rol fueron corregidos directamente en Modelio y no únicamente en los archivos Java.

El código escrito manualmente se mantuvo separado del código producido por Java Designer. No se evaluó la conservación automática de bloques protegidos ni de modificaciones manuales dentro de archivos regenerados.

5.3 Transformación UML a código Java

El proceso realizado corresponde a una transformación Modelo a Texto (M2T), debido a que los elementos estructurados del modelo UML fueron convertidos en archivos textuales con extensión .java.

La correspondencia principal observada se presenta en la Tabla

Elemento UML	Decisión de mapeo aplicada
Clase con «JavaClass»	Se genera una declaración de clase y un archivo NombreClase.java.
Atributo string	Se transforma en un campo Java de tipo String.
Atributo integer	Se transforma en un campo Java de tipo int.
Atributo date	Se transforma en Date con la importación correspondiente.
Operación UML	Se genera un método con nombre, parámetros y tipo de retorno.
Parámetro de dominio	Se utiliza la clase correspondiente, por ejemplo, Mascota mascota, Vacuna vacuna o DocumentoVeterinario documento.
Multiplicidad 0..1 o 1	Se genera una referencia simple a la clase relacionada.
Multiplicidad 0..* o *	Se genera una colección tipada, por ejemplo, List.
Nombre de rol	Se utiliza como nombre del campo asociado; los roles múltiples se definieron en plural.

Agregación composición	o	Se representa estructuralmente mediante referencias o colecciones. La gestión del ciclo de vida no es implementada automáticamente.
---------------------------	---	--

Java Designer produjo declaraciones de clases, atributos, operaciones, referencias y colecciones derivadas de las multiplicidades. El modelo actual no incorpora relaciones de generalización; por ello, el código generado no utiliza extends entre Propietario y Veterinario.

Los cuerpos de varias operaciones permanecieron vacíos, debido a que el generador produce principalmente la estructura inicial de las clases. La lógica de negocio, persistencia, interfaz gráfica y conexión con bases de datos deben implementarse manualmente.

La compilación, ejecución y verificación de los archivos obtenidos se documentan en el capítulo siguiente.

REFERENCIAS

[1] D. C. Schmidt, “Model-Driven Engineering,” IEEE Computer, vol. 39, n.o 2, págs. 25-31, 2006.

[2] M. Brambilla, J. Cabot y M. Wimmer, Model-Driven Software Engineering in Practice. Morgan & Claypool Publishers, 2017.

[3] A. Bucchiarone, J. Cabot, R. F. Paige y A. Pierantonio, “Grand challenges in modeldriven engineering: an analysis of the state of the research,” Software and Systems Modeling, vol. 19, págs. 5-13, 2020. doi: 10.1007/s10270-019-00773-6

[4] O. M. Group, Model Driven Architecture Guide Revision 2.0, OMG Document ormsc/14-06-01, 2014.

[5] Object Management Group, OMG Unified Modeling Language (OMG UML), Version 2.5.1, OMG Document formal/17-12-05, 2017.

[6] Object Management Group. “Meta Object Facility (MOF), Version 2.5.1.” OMG File ID: formal/19-10-01, Object Management Group, visitado 22 de jul. de 2026. dirección: <https://www.omg.org/spec/MOF/2.5.1/PDF>

[7] Object Management Group. “MOF Query/View/Transformation (QVT), Version 1.3.” OMG File ID: formal/16-06-03, Object Management Group, visitado 22 de jul. de 2026. dirección: <https://www.omg.org/spec/QVT/1.3/PDF>

[8] O. M. Group, MOF Model To Text Transformation Language Version 1.0, OMG Document formal/2008-01-16, 2008.

[9] Modelio Open Source. “Modelio Modeling Environment and Main Features,” Modelio Open Source, visitado 22 de jul. de 2026. dirección: <https://github.com/ModelioOpenSource/Modelio>

[10] Modelio Open Source. “Extensions for Modelio: Java Designer,” Modelio Open Source, visitado 22 de jul. de 2026. dirección: <https://github.com/ModelioOpenSource/ExtensionsForModelio>

[11] Modelio Open Source. “Modelio License: GNU General Public License Version 3,” Modelio Open Source, visitado 22 de jul. de 2026. dirección: <https://github.com/ModelioOpenSource/Modelio/blob/master/LICENSE>